

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

1. Hay elecciones municipales en Pythonia y cada mesa de votación cuenta con una urna sumamente tecnológica. Esta urna escanea el voto depositado y escribe la información en un archivo siguiendo el formato: **rut:voto**, donde **rut** es el rut de la persona y **voto** es el código del candidato por el cual vota dicha persona. Ver ejemplo: **votaciones.txt**. Por otra parte, el código y nombre de los candidatos se encuentran en otro archivo, en el que cada línea tiene el formato: **código:nombre**, donde **código** es el código y **nombre** es el nombre del candidato. Ver ejemplo, **candidatos.txt**.

votaciones.txt

```
9173315-3:A1
19642628-0:A2
21933617-9:X27
173315-8:A2
516314-1:C2
```

candidatos.txt

```
A1:Alejandro Pinera
A2:Sebastian Guillier
A3:Beatriz Kast
A4:Leopoldo Sharp
A5:Jorge Mendez
```

Si un elector vota por un candidato que no se encuentra en el archivo de candidatos, el voto se considera nulo.

Desarrolle la función **resultados(vot, can)**, que recibe como parámetros dos *strings* con los nombres de dos archivos con los formatos descritos anteriormente. Esta función debe generar el archivo **resultados.txt**, cuyo encabezado es el total de votos validamente emitidos, es decir, eliminando los votos nulos. Luego está el porcentaje de votos que obtuvo cada candidato, ordenado de mayor a menor. Guíese por el ejemplo a continuación.

Ejemplo:

```
>>> resultados('votaciones.txt', 'candidatos.txt')
```

resultados.txt

```
Total de votos: 3

Sebastian Guillier 66.6 %
Alejandro Pinera 33.3 %
Beatriz Kast 0.0 %
Leopoldo Sharp 0.0 %
Jorge Mendez 0.0 %
```

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

2. Desde el Aeropuerto Internacional de Pythonia salen constantemente vuelos a las principales ciudades del mundo. La información de los vuelos que saldrán en un día en particular se encuentran almacenados en el archivo `salidas-a-m-d.csv`, cuyo nombre incluye el año, mes y día correspondiente. Las líneas del archivo tienen el formato: **vuelo;destino;hora**. El código del vuelo está formado por 2 letras que identifican de manera única a la línea aérea, seguidas de una serie de dígitos. Por ejemplo:

`salidas-2019-08-09.csv`

```
UA7998;Tokyo;12:00
AA6991;Toronto;12:05
VS4667;Chicago;12:20
AA291;Toronto;12:59
AA3492;Chicago;12:20
UA23;Toronto;16:25
AA6250;Madrid;17:05
BA2492;Chicago;17:20
...
```

Escriba la función `agrupar(fecha)`, que recibe como parámetro una tupla de *strings* con formato (aaaa,mm,dd). La función debe crear un archivo llamado `destinos.txt`, que contenga una línea para cada destino distinto encontrado en el archivo de entrada, ordenado de mayor a menor según la cantidad de aerolíneas que vuelan a ese destino.

El formato de cada línea del archivo `destinos.txt` es: **destino:num**, donde **num** corresponde a la cantidad de aerolíneas distintas que vuelan a esa ciudad.

Además de crear el archivo descrito, la función debe retornar el número de destinos distintos que fueron identificados.

Guíese por el ejemplo. En particular, estudie lo que pasa con `Toronto`. A pesar de que hay 3 vuelos a ese destino sólo hay 2 aerolíneas distintas que vuelan ahí, por lo que el archivo de salida indica 2.

Ejemplo:

```
>>> print(agrupar(('2019','08','09')))
```

4

`destinos.txt`

```
Chicago:3
Toronto:2
Tokyo:1
Madrid:1
```

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

3. La Policía de Investigaciones mantiene el registro de todas las personas que han ingresado al país en un archivo que contiene en cada línea un identificador único, el nombre de la persona, su país de procedencia y la fecha de ingreso a territorio nacional. La estructura de cada línea es la siguiente: **identificador:nombre:país:fecha**. La fecha se expresa en el formato **día-mes-año**. Por ejemplo:

inmigrantes.txt

```
456:Cristina Palermo:Argentina:07-05-2019
523:Sophia Watson:EEUU:02-05-2019
364:Alan Moore:EEUU:12-12-2018
111:Karen Lee:EEUU:05-09-2019
646:Nicolás Cabrera:Venezuela:01-01-2019
...
```

La función **filtrar(nombre_archivo, año)** recibe dos parámetros: un **string** con el nombre de un archivo como el anterior, y un **entero** que especifica un año. Esta función retorna un **diccionario** cuyas llaves son los países de procedencia de los inmigrantes en el año ingresado como parámetro. El valor asociado a cada llave es una lista de *strings* con los nombres de las personas de ese país que ingresaron a territorio nacional el año indicado. Estudie el siguiente ejemplo para comprender lo que hace la función:

```
>>> print(filtrar('inmigrantes.txt', 2019))
{'Argentina': ['Cristina Palermo'], 'EEUU': ['Sophia Watson', 'Karen Lee'],
'Venezuela': ['Nicolás Cabrera']}
```

Ordenamiento

A continuación se muestran las instrucciones que forman el código de la función `filtrar`, pero se encuentran desordenadas y sin indentación. Usted debe **ordenarlas** e **indentarlas** adecuadamente para que la función lleve a cabo el proceso descrito. Escriba la función completa a la derecha del código desordenado. No debe agregar nuevas instrucciones y todas las instrucciones dadas deben ser utilizadas.

```
nombre = campos[1]
dicc[pais].append(nombre)
if int(fecha[2]) == año:
dicc[pais] = []
archivo.close()
for linea in archivo:
pais = campos[2]
if pais not in dicc:
archivo = open(nombre_archivo, 'r')
dicc = {}
return dicc
fecha = campos[3].split('-')
def filtrar(nombre_archivo, año):
campos = linea.strip().split(':')
```

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

4. **NOTA:** Puede utilizar la función `filtrar` de la Pregunta 1. No es necesario que la copie de nuevo.

Siguiendo el contexto anterior, escriba la función `contar_ingresos(nombre_archivo, año)` que recibe dos parámetros: un **string** con el nombre de un archivo con el formato indicado, y un **entero** que especifica un año. La función debe retornar un **diccionario** cuyas llaves sean los países de procedencia de los inmigrantes en el año señalado como parámetro. El valor de cada llave debe ser la cantidad de personas de ese país que ingresó a territorio nacional el año indicado.

Ejemplo 1:

```
>>> print(contar_ingresos('inmigrantes.txt', 2019))
{'Argentina': 1, 'EEUU': 2, 'Venezuela': 1}
```

Ejemplo 2:

```
>>> print(contar_ingresos('inmigrantes.txt', 2018))
{'EEUU': 1}
```

5. **NOTA:** Puede utilizar las funciones de las preguntas anteriores. No es necesario que las copie de nuevo.

Siguiendo el contexto anterior, escriba la función `escribir_ingresos(nombre_archivo, año)` que recibe dos parámetros: un **string** con el nombre de un archivo como el descrito, y un **entero** que especifica un año. La función debe crear un archivo de nombre `ingresosX.txt`, donde X corresponde al año ingresado como parámetro. El archivo debe contener los países de procedencia de los inmigrantes en el año especificado en el parámetro, **ordenados** de mayor a menor según cantidad de personas de ese país que ingresaron a territorio nacional ese año. Debe incluir en cada línea su número correlativo partiendo desde el 1. Además, la función debe retornar el número total de personas que ingresaron al país ese año. Guíese por los ejemplos.

Ejemplo 1:

```
>>> print(escribir_ingresos('inmigrantes.txt', 2019))
4
```

ingresos2019.txt

```
1-EEUU:2
2-Venezuela:1
3-Argentina:1
```

Ejemplo 2:

```
>>> escribir_ingresos('inmigrantes.txt', 2018)
1
```

ingresos2018.txt

```
1-EEUU:1
```

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

6. Cesar Pyllan, el encantador de perros, mantiene el registro de las mascotas que ha atendido en un archivo cuya estructura es **fecha;nombre;raza;problema;solucion**. Un ejemplo de este archivo sería:

perros.txt

```
01-01-2015;boby;salchicha;muerde;collar de ahorque
02-03-2016;billy;pastor aleman;miedo;toque al costado
10-12-2016;boby;san bernardo;muerde;toque al costado
24-01-2017;willson;salchicha;miedo;toque al costado
04-02-2017;willson;san bernardo;miedo;toque al costado
03-02-2018;firulais;labrador;estres;SHHH
03-03-2018;firulais;pequines;estres;SHHH
```

La función **leer_perros(na)** se comporta de la siguiente manera:

```
>>> print(leer_perros("perros.txt"))
{'boby': ['collar de ahorque', 'toque al costado'], 'billy': ['toque al costado'],
'willson': ['toque al costado', 'toque al costado'], 'firulais': ['SHHH', 'SHHH']}
```

El código de la función se encuentra desordenado. Usted debe ordenar las líneas e indentarlas correctamente.

```
_,n,r,p,s = l.strip().split(';')
pe[n].append(s)
return pe
a = open(na)
a.close()
def leer_perros(na):
pe[n] = []
for l in a:
if n not in pe:
pe = {}
```

Finalmente, explique en menos de 30 palabras lo que realiza la función anterior.

Programación – Ejercicios sobre Procesamiento de Texto y Archivos

7. **NOTA:** Considere el contexto de la pregunta anterior. Puede utilizar la función `leer_perros(na)` de esa pregunta.

En esta oportunidad César Pyllan le solicita su ayuda para lo siguiente:

a.- Implemente la función `leer_razas(nombre_archivo)`, que recibe un *string* con el nombre de un archivo y clasifica los animales por raza. La función retorna un diccionario cuya llave es el nombre de la raza y el valor una lista con los nombres de los perros que pertenecen a ella. Guíese por el ejemplo a continuación.

Ejemplo

```
>>> print(leer_razas('perros.txt'))
{'salchicha': ['boby', 'willson'], 'pastor aleman': ['billy'],
'san bernardo': ['boby', 'willson'], 'labrador': ['firulais'],
'pequines': ['firulais']}
```

b.- César ha notado que existen perros con el mismo nombre en distintas razas. Sabiendo que los nombres de los perros son únicos, esto sólo puede significar que existen mestizos. Desarrolle una función llamada `mestizos(razal, raza2, nombre_archivo)` que recibe dos razas de perro y el nombre del archivo. Esta función debe retornar una lista con los nombres de los perros mestizos de dichas razas. Si no existen perros mestizos para las razas consultadas, la función debe retornar una lista vacía. Guíese por los ejemplos a continuación.

Ejemplos

```
>>> print(mestizos('salchicha', 'san bernardo', 'perros.txt'))
['boby', 'willson']
>>> print(mestizos('pequines', 'labrador', 'perros.txt'))
['firulais']
>>> print(mestizos('pequines', 'pastor aleman', 'perros.txt'))
[]
```

c.- Dado que los casos de perros mestizos aumentan, Cesar debe tener soluciones rápidas. Desarrolle la función `solucion_mestizos(razal, raza2, nombre_archivo)` la que, dadas dos razas y el nombre del archivo de los perros, retorna un diccionario cuya llave es la solución y el valor asociado es la cantidad de perros distintos a los que fue recomendada dicha solución, considerando solo a los mestizos de las razas consultadas. Si a un perro le fue recomendada una solución varias veces, debe ser contado una única vez.

Ejemplos

```
>>> print(solucion_mestizos('salchicha', 'san bernardo', 'perros.txt'))
{'collar de ahorque': 1, 'toque al costado': 2}
>>> print(solucion_mestizos('pequines', 'labrador', 'perros.txt'))
{'SHHH': 1}
>>> print(solucion_mestizos('pequines', 'pastor aleman', 'perros.txt'))
{}
```